

Rekurencja & iteracja

Pzyp. $\mathbb{N}_0$ oznaczamy zbiór liczb naturalnych **wraz z zerem**

1. Wersja rekurencyjna silni, funkcja $S(n \in \mathbb{N}_0) \rightarrow \mathbb{N}$. Wypisać stos wywołań dla $S(5)$:

1. $S(n) := 1$, gdy $n \leq 1$

2. $S(n) := n \cdot S(n - 1)$, w każdym innym wypadku.

2. Wersja rekurencyjna n -tego wyrazu ciągu Fibonacciego, funkcja $F(n \in \mathbb{N}_0) \rightarrow \mathbb{N}$ (czyli pierwszy wyraz to $F(0)$). Zwizualizować drzewo rekurencji oraz stopniowe obliczanie poszczególnych zagnieżdżonych wywołań.

1. $F(n) := 0$, gdy $n = 0$

2. $F(n) := 1$, gdy $n = 1$

3. $F(n) := F(n - 1) + F(n - 2)$, gdy $n \geq 2$.

3. Wersja rekurencyjna szukania maksimum w tablicy 1-wymiarowej, k -elementowej. Funkcja $rmax(A \in \mathbb{N}^k, start, end \in \mathbb{N}_0) \rightarrow \mathbb{N}$, gdzie A to tablica wejściowa, $start$ to indeks od którego zaczynamy szukać maksimum, a end to ostatni indeks zakresu, jaki przeszukujemy.

1. Wewnątrz funkcji, mamy wykorzystać podfunkcję $max(x, y \in \mathbb{N}) \rightarrow \mathbb{N}$, do odnajdywania większej z dwóch liczb naturalnych.

2. Zatem aby znaleźć maksimum np. w całej tablicy $A \in \mathbb{N}^5$, poprawne wywołanie to $rmax(A, 0, 4)$. Przy kolejnych wywołaniach wewnątrz funkcji, powinniśmy zwiększyć $start$ o jeden (lub zmniejszyć end o jeden, w zależności od której strony chcemy szukać).

3. Jako warunek końcowy możemy przyjąć długość przeszukiwanego zakresu równą 1, zwracamy wtedy $A[start]$ (ekwiwalentnie, $A[end]$, bo wtedy $start = end$).

4. Tak jak w 1: **zwizualizować przykładowy stos wywołań** dla $A = \{3, 4, 2, 1\}$.
Zaczynamy tutaj zatem od $call\ rmax(\{3, 4, 2, 1\}, 0, 3)$.

4. Używając znanego wzoru na położenie elementu i, j w tablicy $A \in \mathbb{N}^{w,k}$ w odpowiadającej tablicy jednowymiarowej $B \in \mathbb{N}^{w \cdot k}$ (np. $\mathbb{N}^{3,4} \rightarrow \mathbb{N}^{12}$) uporządkowanej według zasady **row-major**, przyp.: mamy takie $l := i \cdot k + j$, aby $B[l]$ było równe $A[i][j]$: zaproponować **rekurencyjną wersję szukania maksimum w tablicy 2-wymiarowej** (posługując się pomocniczą tablicą 1D według sugerowanego schematu).

1. *do samodzielnej analizy* -- co jeśli chcemy otrzymać $\arg \max$ według tej metody, to jest położenie w oryginalnej tablicy 2-wymiarowej? Jak wrócić z l do i, j ? *warto rozważyć dzielenie w liczbach całkowitych i podobne sprawy.*

5. Wersja rekurencyjna n -tego wyrazu ciągu Tribonacciego. Funkcja $T(n \in \mathbb{N}_0) \rightarrow \mathbb{N}$. Zwizualizować drzewo rekurencji i stopniowe obliczanie zagnieżdżonych wywołań jak w 2, dla $T(5)$:

1. $T(n) := 0$, gdy $n \leq 1$

2. $T(n) := 1$, gdy $n = 2$

3. $T(n) := T(n-1) + T(n-2) + T(n-3)$, gdy $n \geq 3$.

6. **Optymalizacje cz.I** Wersja iteracyjna algorytmów:

1. na silnię

2. na n -ty wyraz ciągu Fibonacciego

3. na n -ty wyraz ciągu Tribonacciego. Dla Fibonacciego -- wykonać śledzenie wykonania pętli w tabeli dla $F(5)$.

7. **Optymalizacje cz.II** Wersja "z pamięcią" (*memoizacja*) funkcji liczącej n -ty wyraz ciągu Fibonacciego, funkcja $Fm(n \in \mathbb{N}_0, C \in \mathbb{N}^{n+1}) \rightarrow \mathbb{N}$. ("*Fibonacci memory*").

1. C to tablica na tw. `cache` - pamięć podręczną.

2. Przed pierwszym uruchomieniem, wypełniamy ją liczbami -1 .

3. w kolejnych uruchomieniach funkcji -- jeśli już możemy, zastępujemy $C[i]$ dotychczas obliczonym wynikiem $Fm(i, C)$.

4. no i jeśli już $C[i] \neq -1$, nie liczymy dalej tylko zwracamy $C[i]$.

8. **Optymalizacje cz.IIIa** (uwaga - efektywność stosowania akurat tej techniki jest ciężko zauważyć w przypadku języka C) Wersja "ogonowa" rekurencyjnej funkcji liczącej silnię: funkcja $S_acc(n \in \mathbb{N}_0, ak \in \mathbb{N}) \rightarrow \mathbb{N}$, gdzie: n to liczba, której silnię chcemy policzyć, ak to "akumulator" - pomocnicza zmienna do trzymania dotychczasowych wyników mnożenia. Zwizualizować stos wywołań (*call stack*) także dla tego przypadku dla $S_acc(5, 1)$.

9. **Optymalizacje cz.IIIb** Wersja "ogonowa" szukania maksimum w tablicy jednowymiarowej z pkt.3: $rmax_acc(A \in \mathbb{N}^k, start, end \in \mathbb{N}_0, ak \in \mathbb{N}) \rightarrow \mathbb{N}$. Pierwsze wywołanie dla $A \in \mathbb{N}^5$ wyglądałoby jak $rmax_acc(A, 0, 4, A[0])$ - jeśli szukamy "od lewej"; lub

$rmax_acc(A, 0, 4, A[4])$ - jeśli szukamy od prawej. **Wskazówka:** wcześniej ak (w zadaniu o silni) był update'owany wynikiem binarnej operacji - mnożenia. Teraz, jeśli nie szukamy iloczynu liczb, tylko ich maksimum, to ta operacja musi być czym?

Do domu: jeśli chcemy zrozumieć korzyści danej optymalizacji - dobrze zaimplementować wybrane funkcje w języku programowania i dołożyć np. instrukcje print wewnątrz funkcji lub licznik wywołań.